

Computação Além dos Dados

Restrições, Acessibilidade e Memória como
Fundamentos de Sistemas Complexos

Flyxion

Abstract

Grande parte da Ciência da Computação moderna é construída sobre uma suposição aparentemente óbvia: primeiro existem os dados, depois os algoritmos operam sobre eles. Este ensaio propõe uma inversão parcial dessa perspectiva. Em vez de tratar objetos, dados ou estados como elementos fundamentais, o foco passa para as restrições, trajetórias possíveis e espaços de acessibilidade que tornam certos estados alcançáveis e outros impossíveis. Essa mudança de ênfase não é apenas filosófica. Ela tem consequências diretas para a forma como projetamos sistemas de IA, modelamos memória computacional, entendemos arquiteturas distribuídas e concebemos a cognição. O ensaio apresenta cinco frameworks interconectados — CLIO, RSVP, MEM|8, HYDRA e TARTAN — como instâncias de um programa teórico unificado: a ideia de que restrição precede conteúdo, que acessibilidade é mais fundamental que distância, e que a pergunta mais produtiva sobre qualquer sistema complexo não é *o que existe?* mas *o que ainda é possível?*

Contents

1	A Suposição Invisível	3
2	O Princípio Central: Restrição Antes do Conteúdo	3
2.1	O Jogo de Xadrez	3
2.2	A Proposição Central	4
2.3	Restrições como Informação	4
3	CLIO: Inteligência como Navegação em Espaços de Projeção	5
3.1	O Problema da Observação Parcial	5
3.2	Incerteza como Estrutura	5
3.3	Inteligência como Redução Eficiente de Ambiguidade	6
3.4	Falha de Projeção e Paradoxo	6
4	RSVP: Sistemas Complexos como Processos de Relaxamento	6
4.1	Da Cosmologia à Computação	6
4.2	Potencial Institucional e Descida de Gradiente	7
4.3	Atratores e Configurações Estáveis	7
4.4	Entropia, Informação e Geometria	8
5	MEM 8: Memória como Trajetória, não como Estado	8
5.1	O Modelo Convencional de Memória	8
5.2	MEM 8: Memória como Registro de Colapsos	9
5.3	Conexões com Tecnologias Existentes	9
5.4	Memória de Onda e Persistência	10
6	HYDRA: Cognição como Coordenação Modular	10
6.1	O Problema da Arquitetura Cognitiva	10
6.2	HYDRA: Múltiplos Módulos Especializados	10
6.3	Conexões com Arquiteturas Modernas de IA	11
7	TARTAN: Decomposição que Preserva Significado	11
7.1	O Problema da Perda de Contexto	11
7.2	Decomposição Estrutural Preservadora	12
7.3	TARTAN e Admissibilidade	12
8	Acessibilidade em Vez de Distância	13
8.1	A Mudança de Pergunta	13
8.2	Volume de Acessibilidade	13
8.3	Liberdade como Volume de Acessibilidade	14

9 IA como Motor de Simulação	14
9.1 Percepção como Inferência	14
9.2 Memória como Reconstrução	15
9.3 Raciocínio como Simulação	15
9.4 Consciência como Projeção e Correção	15
10 O Núcleo Unificado: Admissibilidade	16
10.1 O Conceito Central	16
10.2 Admissibilidade como Geometria	16
10.3 A Pergunta Unificada	17
11 Implicações para a Prática	17
11.1 Design de Sistemas	17
11.2 Avaliação de Sistemas de IA	17
11.3 Engenharia de Memória	18

A Suposição Invisível

Todo estudante de Ciência da Computação aprende, cedo, a pensar em termos de dados e operações. Existe um array; aplicamos um algoritmo de ordenação. Existe um grafo; executamos uma busca em largura. Existe um conjunto de exemplos rotulados; treinamos um classificador. A estrutura geral é sempre a mesma: o mundo fornece objetos, e nós os processamos.

Essa suposição é tão fundamental que raramente é examinada. Ela aparece na terminologia básica — variável, valor, estado, dado — e organiza silenciosamente a forma como pensamos sobre quase tudo na área.

Mas existe uma outra forma de pensar. Em vez de começar pelos objetos e perguntar o que podemos fazer com eles, podemos começar pelas restrições e perguntar quais objetos elas tornam possíveis. Em vez de começar pelos estados e perguntar como transicionamos entre eles, podemos começar pelos espaços de estados e perguntar quais regiões são acessíveis a partir de onde estamos.

Essa inversão não é apenas uma curiosidade filosófica. Ela produz perguntas diferentes, modelos diferentes, e, argumentamos aqui, uma compreensão mais profunda de vários dos problemas mais difíceis da computação moderna: o problema da generalização em aprendizado de máquina, o problema da consistência em sistemas distribuídos, o problema da persistência em memória computacional, o problema da emergência em sistemas cognitivos.

O objetivo deste ensaio é apresentar um conjunto de frameworks teóricos que partem dessa inversão e exploram suas consequências em diferentes domínios. Os frameworks são independentes o suficiente para serem estudados separadamente, mas compartilham um núcleo conceitual comum que os torna partes de um programa teórico unificado.

O Princípio Central: Restrição Antes do Conteúdo

O Jogo de Xadrez

Considere um jogo de xadrez. As peças importam, mas as regras que definem os movimentos possíveis importam ainda mais. Sem regras, não existe jogo — existe apenas um conjunto de objetos de madeira ou plástico sobre uma superfície quadriculada. O jogo emerge das restrições, não dos componentes.

Isso não é trivial. Significa que a identidade de uma peça — o que ela *é*, funcionalmente — é completamente determinada pelas restrições que governam seu comportamento. Um rei que pudesse se mover como uma rainha não seria mais um rei. Trocar a regra é trocar o objeto, mesmo que a peça física permaneça a mesma.

A mesma lógica aparece em toda a computação:

Em compiladores, a gramática formal define o espaço de programas sintática-

mente válidos. Um programa não é apenas uma sequência de caracteres; é uma sequência que satisfaz as restrições impostas pela gramática. A gramática precede e constitui o programa.

Em bancos de dados relacionais, o esquema define quais registros podem existir. Um banco de dados não é apenas uma coleção de dados; é uma coleção de dados que satisfaz um conjunto de restrições de integridade. Violar essas restrições não é apenas um erro técnico; é uma contradição com a própria definição do que o banco de dados representa.

Em redes neurais, a arquitetura define o espaço de funções que a rede pode aprender. Antes de ver qualquer exemplo, a arquitetura já determina quais representações são possíveis e quais são impossíveis. A restrição arquitetural precede os dados de treinamento.

Em sistemas operacionais, o modelo de permissões define quais ações cada processo pode executar. O conceito de segurança não é um add-on adicionado após o sistema ser construído; é a estrutura de restrições sobre a qual o sistema opera.

A Proposição Central

Princípio de Restrição. As restrições que definem um sistema não são detalhes de implementação adicionados para evitar erros. São a principal fonte de organização do sistema — a estrutura que determina quais estados existem, quais transições são possíveis, e quais objetivos podem ser perseguidos.

Essa perspectiva tem uma consequência imediata: para entender um sistema complexo, a primeira pergunta não é *quais elementos ele contém?* mas *quais restrições o organizam?*

Restrições como Informação

Há uma conexão profunda entre restrições e informação que vale a pena tornar explícita. Shannon definiu informação como a quantidade que reduz incerteza. Uma mensagem é informativa na medida em que elimina alternativas possíveis.

Restrições fazem exatamente isso. Uma restrição elimina estados possíveis. Quanto mais restritivo é um sistema, menor é o espaço de estados acessíveis — e, portanto, mais informação está implicitamente codificada na estrutura de restrições.

Essa observação tem implicações práticas. Quando projetamos um sistema, cada decisão de design que reduz o espaço de estados possíveis é também uma decisão que codifica informação sobre o propósito do sistema. Um sistema sem restrições é um sistema sem propósito — e um sistema sem propósito é computacionalmente inexpressivo.

Proposição 1 (Equivalência Restrição-Informação). *Seja S o espaço completo de estados de um sistema e C o subconjunto de estados admissíveis sob um conjunto de restrições R . A quantidade de informação codificada por R é proporcional a $\log |S| - \log |C|$: a diferença entre o espaço irrestrito e o espaço restrito. Restrições mais fortes codificam mais informação.*

CLIO: Inteligência como Navegação em Espaços de Projeção

O Problema da Observação Parcial

Sistemas inteligentes raramente observam a realidade completa. Eles recebem projeções parciais do mundo: pixels em vez de objetos, tokens em vez de significados, sinais sensoriais em vez de estados físicos.

Formalmente, seja X o espaço completo de estados do mundo e M o espaço de observações disponíveis para o sistema. Existe uma função de projeção:

$$\pi : X \rightarrow M$$

que mapeia estados do mundo em observações. Em geral, π não é injetora: múltiplos estados do mundo podem produzir a mesma observação.

O problema central da inteligência, nessa perspectiva, é a inversão dessa projeção: dado $m \in M$, qual é o conjunto

$$\pi^{-1}(m) = \{x \in X \mid \pi(x) = m\}$$

de estados do mundo consistentes com a observação?

Incerteza como Estrutura

A pré-imagem $\pi^{-1}(m)$ representa a incerteza estrutural do sistema: a ambiguidade que é matematicamente inerente à projeção, independente de qualquer limitação de processamento. Um sistema que observa m não pode distinguir entre nenhum dos estados em $\pi^{-1}(m)$ apenas com base nessa observação.

Isso tem várias consequências importantes:

Para aprendizado de máquina: o problema de generalização é fundamentalmente o problema de inverter uma projeção. Um modelo treinado em exemplos viu apenas uma amostra do espaço de estados do mundo. A generalização é a capacidade de reconstruir a estrutura do espaço de estados a partir dessas amostras.

Para sistemas de percepção: a ambiguidade perceptual não é um bug que pode ser eliminado com mais dados ou melhor hardware. É uma propriedade estrutural da relação entre sensores e mundo.

Para raciocínio sob incerteza: a incerteza bayesiana é uma forma de representar $\pi^{-1}(m)$ através de distribuições de probabilidade sobre estados do mundo.

Inteligência como Redução Eficiente de Ambiguidade

O framework CLIO define inteligência operacionalmente como a capacidade de reduzir o espaço $\pi^{-1}(m)$ de maneira eficiente, usando restrições adicionais derivadas do contexto, da memória e do raciocínio.

Definição 1 (Inteligência CLIO). *Um sistema é inteligente na medida em que, dado $m \in M$, consegue identificar um subconjunto $\mathcal{A}(m) \subseteq \pi^{-1}(m)$ de estados plausíveis tal que $|\mathcal{A}(m)| \ll |\pi^{-1}(m)|$, usando restrições derivadas de contexto e memória, sem acesso direto a X .*

Essa definição unifica vários conceitos que normalmente aparecem separados em IA:

Raciocínio é a aplicação de restrições lógicas para reduzir $\pi^{-1}(m)$.

Aprendizado é a aquisição de restrições que tornam a redução mais eficiente em novos contextos.

Memória é o armazenamento de restrições derivadas de experiências anteriores.

Generalização é a transferência de restrições para contextos não observados.

Falha de Projeção e Paradoxo

Uma consequência importante do framework CLIO é uma explicação natural para certos tipos de paradoxo e ilusão cognitiva.

Definição 2 (Falha de Projeção). *Uma falha de projeção ocorre quando um sistema trata a projeção m como se fosse o estado completo x — quando confunde o mapa com o território.*

Muitos erros sistemáticos de raciocínio humano e de sistemas de IA podem ser entendidos como falhas de projeção. Vieses cognitivos surgem quando o sistema usa heurísticas que funcionam bem para a maioria dos estados em $\pi^{-1}(m)$ mas falham para estados menos típicos. Alucinações em LLMs surgem quando o modelo trata sua representação interna como se fosse o estado real do mundo.

Paradoxos semânticos surgem quando uma descrição m é tratada como referência direta a um estado x , quando na verdade $\pi^{-1}(m)$ contém múltiplos estados com propriedades incompatíveis. Ao restaurar o contexto — ao especificar em qual estado de $\pi^{-1}(m)$ estamos — o paradoxo frequentemente dissolve.

RSVP: Sistemas Complexos como Processos de Relaxamento

Da Cosmologia à Computação

O framework RSVP foi desenvolvido originalmente como uma teoria cosmológica alternativa, mas sua estrutura matemática possui uma interpretação computacional

direta e independente da motivação original.

A ideia central é que sistemas complexos evoluem para estados que reduzem tensões estruturais internas. Em vez de imaginar um sistema como um conjunto estático de objetos em estados fixos, RSVP o descreve como um processo dinâmico de reorganização contínua de restrições.

Potencial Institucional e Descida de Gradiente

Formalmente, seja $x(t)$ o estado de um sistema no tempo t e $V(x, t)$ uma função potencial que representa tensões estruturais internas — inconsistências, restrições violadas, gradientes não resolvidos. O sistema evolui aproximadamente por descida de gradiente:

$$\frac{dx}{dt} = -\nabla_x V(x, t) + \xi(t),$$

onde $\xi(t)$ representa perturbações externas.

Cientistas da computação reconhecerão essa estrutura imediatamente: é a mesma de descida de gradiente estocástica em aprendizado de máquina, de otimização variacional em modelos generativos, e de equações de difusão em sistemas físicos.

A diferença de perspectiva é que RSVP trata a função V não como uma função objetivo escolhida pelo projetista, mas como uma propriedade emergente da estrutura de restrições do sistema. A “otimização” é interna e não requer um objetivo externo explícito.

Atratores e Configurações Estáveis

Um atrator é uma região do espaço de estados para a qual o sistema converge naturalmente. Na perspectiva RSVP, atratores correspondem a configurações que minimizam tensões estruturais localmente.

Proposição 2 (Atratores como Mínimos de Tensão). *As configurações estáveis de um sistema complexo correspondem a mínimos locais da função de tensão $V(x, t)$. Uma perturbação desestrutura o sistema ao aumentar V além de um limiar crítico, tornando o mínimo local instável — quando a Hessiana $\nabla_x^2 V(x^*, t)$ perde positividade definida.*

Esse framework tem aplicações diretas para entender:

Treinamento de redes neurais: o processo de treinamento é a busca por mínimos de uma função de perda. A paisagem de perda tem múltiplos mínimos locais, e diferentes configurações de hiperparâmetros correspondem a diferentes funções de tensão.

Sistemas distribuídos: a consistência eventual em sistemas distribuídos pode ser entendida como convergência para um atrator global a partir de estados localmente inconsistentes.

Modelos de linguagem: a geração de texto pode ser vista como navegação em um espaço de estados linguísticos com uma função de tensão implícita definida pelos dados de treinamento.

Entropia, Informação e Geometria

Uma das contribuições conceituais mais ricas de RSVP é a reinterpretação de entropia e informação como propriedades geométricas da dinâmica do sistema.

Em vez de tratar entropia como uma quantidade escalar associada a uma distribuição de probabilidade, RSVP a trata como uma densidade de campo: uma quantidade que varia no espaço e no tempo e que codifica tensões estruturais locais.

Essa perspectiva conecta-se naturalmente a:

Inferência variacional: que minimiza uma divergência entre uma distribuição aproximada e uma distribuição alvo — interpretável como minimização de tensão em um espaço de distribuições.

Normalizing flows: que aprendem transformações que mapeiam distribuições simples em distribuições complexas, preservando a estrutura de volume — uma forma de navegação em espaços de configuração.

Geometria da informação: que estuda a curvatura do espaço de distribuições de probabilidade — análoga à curvatura do espaço de configurações em RSVP.

MEM | 8: Memória como Trajetória, não como Estado

O Modelo Convencional de Memória

O modelo convencional de memória computacional é simples e eficiente: memória é um mapa de endereços para valores. Ler memória é recuperar um valor. Escrever memória é substituir um valor por outro. O histórico de escritas não é preservado; apenas o estado atual importa.

Esse modelo é enormemente poderoso para a maioria das aplicações. Mas ele tem limitações estruturais que se tornam visíveis em certos contextos:

Em sistemas distribuídos, dois processos podem ter escrito valores diferentes no “mesmo” endereço. O estado atual não é suficiente para resolver o conflito; é necessário conhecer a ordem das escritas.

Em sistemas de controle de versão como Git, o estado atual de um repositório não é suficiente para entender como ele chegou lá. O histórico de commits é uma parte essencial da representação.

Em bancos de dados de auditoria, é necessário preservar não apenas o estado atual mas a sequência de todas as modificações.

Em cognição, a identidade não é apenas o estado atual de um organismo mas a continuidade de sua trajetória através do tempo.

MEM | 8: Memória como Registro de Colapsos

O framework MEM | 8 propõe uma inversão do modelo convencional. Em vez de tratar memória como armazenamento de estados, trata-a como registro de decisões — mais precisamente, como registro dos colapsos sucessivos de espaços de possibilidade.

Definição 3 (Memória MEM | 8). *Uma memória MEM | 8 é uma sequência*

$$\mu = (e_1, e_2, \dots, e_n)$$

de eventos, onde cada evento e_i representa uma decisão que colapsou um espaço de possibilidades $\mathcal{P}_{i-1}$ em um espaço reduzido $\mathcal{P}_i \subseteq \mathcal{P}_{i-1}$. O estado atual é

$$s_n = \bigcap_{i=1}^n \mathcal{C}(e_i),$$

onde $\mathcal{C}(e_i)$ é o conjunto de estados consistentes com o evento e_i .

Essa formulação tem várias consequências:

Identidade é continuidade: a identidade de um sistema não é seu estado atual, mas a trajetória que o produziu. Dois sistemas no mesmo estado podem ter identidades diferentes se chegaram lá por trajetórias diferentes.

Persistência é dinâmica: a persistência de uma entidade não é um estado estático mas um processo ativo de manutenção da continuidade da trajetória.

Recuperação é reconstrução: recuperar uma memória não é acessar um valor armazenado mas reconstruir um estado a partir da sequência de eventos que o produziram.

Conexões com Tecnologias Existentes

O framework MEM | 8 tem conexões explícitas com várias tecnologias e paradigmas já estabelecidos na engenharia de software:

Event Sourcing: um padrão arquitetural onde o estado de um sistema é derivado de uma sequência imutável de eventos. O estado atual é uma projeção do log de eventos, não um valor armazenado independentemente.

CRDTs (Conflict-free Replicated Data Types): estruturas de dados que garantem consistência eventual em sistemas distribuídos ao definir operações de merge que comutam e são idempotentes — análogo a sistemas onde a ordem dos eventos não altera o estado final.

Blockchains: registros distribuídos imutáveis onde o estado atual é completamente determinado pelo histórico de transações. A imutabilidade do histórico é a garantia de integridade.

Bancos de dados temporais: bancos que preservam o histórico completo de modificações, permitindo queries sobre estados passados.

Sistemas de controle de versão: como Git, onde a identidade de um repositório é sua árvore de commits, não apenas o estado atual dos arquivos.

Memória de Onda e Persistência

Uma extensão do framework MEM | 8 trata a memória não como uma sequência discreta de eventos mas como um campo contínuo de influências — uma “memória de onda” onde eventos passados irradiam influência que decai com o tempo mas nunca desaparece completamente.

Essa formulação é relevante para:

Redes neurais recorrentes: onde o estado oculto codifica uma forma de memória de curto prazo que influencia continuamente o processamento atual.

Mecanismos de atenção: que permitem que modelos de linguagem acessem informação de contextos anteriores com pesos que dependem da relevância.

Memória episódica em IA: sistemas que armazenam experiências completas e as recuperam por similaridade com o contexto atual.

HYDRA: Cognição como Coordenação Modular

O Problema da Arquitetura Cognitiva

Como organizar um sistema inteligente? A resposta mais simples é um sistema monolítico: um único modelo que recebe entradas, processa internamente, e produz saídas. Essa arquitetura é matematicamente elegante e computacionalmente tratável.

Mas ela tem limitações. Sistemas monolíticos são difíceis de inspecionar, difíceis de depurar, difíceis de estender, e difíceis de adaptar a domínios muito diferentes do domínio de treinamento. Acima de certo tamanho, tornam-se caixas-pretas opacas cujo comportamento é difícil de prever ou garantir.

HYDRA: Múltiplos Módulos Especializados

O projeto HYDRA parte da hipótese de que cognição robusta emerge da coordenação entre módulos especializados, não de um único mecanismo monolítico. Os módulos são:

Percepção: transforma inputs sensoriais em representações internas. Não é uma passagem direta de dados; é uma projeção ativa que enfatiza certas dimensões e suprime outras.

Memória: mantém representações de estados passados e os torna disponíveis para outros módulos. No framework MEM | 8, isso é o registro da trajetória.

Planejamento: projeta futuros possíveis a partir do estado atual e das restrições disponíveis. Fundamentalmente, é navegação no espaço de estados acessíveis.

Coordenação: garante que os módulos operam coerentemente como um sistema. É o módulo que gerencia as interações e resolve conflitos entre as saídas dos outros módulos.

Monitoramento: avalia o desempenho do sistema como um todo e detecta quando outros módulos estão falhando ou operando fora de seus domínios de competência.

Integração: produz respostas coerentes combinando as contribuições dos outros módulos.

Proposição 3 (Emergência Cognitiva). *Em sistemas HYDRA, comportamentos cognitivos complexos emergem da coordenação entre módulos simples sem que nenhum módulo individual possua esses comportamentos. A coordenação é a fonte da complexidade, não a sofisticação de cada módulo individualmente.*

Conexões com Arquiteturas Modernas de IA

A hipótese central de HYDRA — que cognição emerge de coordenação modular — tem paralelos com várias arquiteturas modernas de IA:

Mixture of Experts (MoE): modelos onde diferentes “especialistas” são ativados para diferentes inputs, com um mecanismo de roteamento que decide qual especialista usar.

Sistemas multiagente: arquiteturas onde múltiplos agentes cooperam ou competem para resolver problemas coletivos.

Modelos de atenção multi-cabeça: onde diferentes “cabeças” de atenção capturam diferentes tipos de dependências no input.

Sistemas neurossimbólicos: que combinam redes neurais com raciocínio simbólico, tipicamente através de módulos especializados para cada paradigma.

A diferença de HYDRA é que torna a coordenação explícita e tratável, em vez de deixá-la emergir implicitamente do treinamento.

TARTAN: Decomposição que Preserva Significado

O Problema da Perda de Contexto

Um problema central em sistemas distribuídos e em IA é a perda de contexto durante transformações. Quando informação é dividida, resumida, comprimida ou transmitida entre componentes, parte do significado original desaparece.

Formalmente, isso é a perda de informação em projeções:

$$H(X) - H(\pi(X)) > 0,$$

onde H é entropia de Shannon. Qualquer projeção que não seja bijetora perde informação.

O problema não é eliminar essa perda — que frequentemente é necessária para escalabilidade e eficiência — mas controlá-la: garantir que as informações mais importantes para o propósito do sistema sejam preservadas.

Decomposição Estrutural Preservadora

O framework TARTAN investiga como decompor sistemas complexos de maneira que certas propriedades importantes sejam preservadas ao longo das transformações.

Definição 4 (Decomposição TARTAN). *Uma decomposição D de um sistema S é uma decomposição TARTAN para um conjunto de propriedades P se, para toda propriedade $p \in P$:*

$$p(S) = p(D(S)),$$

isto é, a propriedade é idêntica no sistema original e na decomposição.

As propriedades que o sistema quer preservar dependem do contexto:

Em processamento distribuído, a propriedade pode ser a consistência eventual — garantir que todas as réplicas convergem para o mesmo estado.

Em IA explicável (XAI), a propriedade pode ser a fidelidade da explicação — garantir que a explicação do comportamento do modelo é uma representação precisa do que o modelo realmente calcula.

Em compressão semântica, a propriedade pode ser a preservação de relações de inferência — garantir que o que é inferível da versão comprimida é exatamente o que seria inferível da versão completa.

Em sistemas multiagente, a propriedade pode ser a coordenação — garantir que agentes que recebem partes diferentes de uma tarefa produzem resultados que se combinam coerentemente.

TARTAN e Admissibilidade

Há uma conexão profunda entre TARTAN e o conceito de admissibilidade que percorre todos os frameworks apresentados.

Uma decomposição TARTAN para um conjunto de propriedades P é, essencialmente, uma forma de garantir que as transformações aplicadas ao sistema permanecem dentro do espaço de transformações admissíveis — o espaço de transformações que preservam as propriedades relevantes.

Isso conecta TARTAN a:

Teoria de categorias: que estuda transformações que preservam estrutura (funtores, transformações naturais).

Homomorfismos: em álgebra, transformações que preservam operações.

Bisimulações: em teoria de concorrência, relações de equivalência entre sistemas que preservam comportamento observável.

Acessibilidade em Vez de Distância

A Mudança de Pergunta

Uma das contribuições conceituais mais originais do programa teórico apresentado aqui é a substituição da pergunta *quão longe está algo?* pela pergunta *quão difícil é chegar até isso?*

Na computação, essa mudança é fundamental:

Um arquivo pode estar armazenado no mesmo disco físico que um processo e ser completamente inacessível a esse processo, seja por permissões, por corrupção de metadados, ou por falha no sistema de arquivos.

Um dado pode existir em um banco de dados distribuído e ser irrecuperável devido a falhas de rede ou partições.

Um estado pode ser teoricamente alcançável em um sistema de controle mas computacionalmente inviável dado o poder de processamento disponível.

Uma solução pode existir para um problema de otimização mas estar em uma região do espaço de busca que os algoritmos disponíveis não conseguem explorar eficientemente.

Em todos esses casos, a distância física ou lógica entre o estado atual e o estado desejado é irrelevante. O que importa é a *acessibilidade*: a existência de um caminho admissível entre os dois estados.

Volume de Acessibilidade

Definição 5 (Volume de Acessibilidade). *Para um sistema em estado x , o conjunto de acessibilidade $R(x)$ é o conjunto de todos os estados alcançáveis a partir de x por sequências de transições admissíveis. O volume de acessibilidade é*

$$\Omega(x) = \text{Vol}(R(x)).$$

O volume de acessibilidade é uma medida da *liberdade* do sistema — o quão amplo é o espaço de futuros possíveis a partir do estado atual. Sistemas com volume de acessibilidade alto têm muitas opções; sistemas com volume baixo estão essencialmente determinados.

Essa métrica aparece, sob diferentes nomes, em várias áreas:

Teoria da complexidade computacional: a classe de problemas que um algoritmo pode resolver eficientemente é seu espaço de acessibilidade computacional.

Alcançabilidade em sistemas de controle: o conjunto de estados que um sistema de controle pode atingir a partir de um estado inicial dado é seu conjunto de acessibilidade.

Grafos de estados em model checking: a verificação de propriedades de sistemas concorrentes envolve explorar o grafo de estados acessíveis.

Espaços de busca em planejamento de IA: algoritmos de planejamento navegam em grafos de estados buscando sequências de ações que atingem um estado objetivo.

Liberdade como Volume de Acessibilidade

Há uma conexão importante entre acessibilidade e liberdade que vai além da computação.

Proposição 4 (Reachability Principle — versão computacional). *A liberdade efetiva de um agente é proporcional não aos recursos que possui mas ao volume de estados futuros acessíveis a partir de seu estado atual:*

$$\text{Liberdade}(x) \propto \Omega(x).$$

Essa proposição tem implicações práticas imediatas para o design de sistemas:

Um sistema de software que reduz progressivamente o espaço de estados acessíveis para seus usuários — ao acumular estado interno, ao criar dependências de dados, ao restringir opções de configuração — está reduzindo a liberdade efetiva dos usuários, independente de qualquer enriquecimento de funcionalidades.

Um sistema de IA que converge para uma distribuição de saídas cada vez mais estreita à medida que processa mais contexto está reduzindo seu próprio volume de acessibilidade — potencialmente de maneira irreversível.

IA como Motor de Simulação

Percepção como Inferência

A hipótese de que sistemas inteligentes funcionam como motores de simulação — construindo modelos internos do mundo que permitem prever futuros possíveis — tem raízes profundas na ciência cognitiva e conecta-se diretamente com os frameworks apresentados.

Na perspectiva do framework CLIO, percepção não é a passagem direta de dados do ambiente para o sistema. É um processo de inferência: dado o sinal sensorial $m = \pi(x)$, o sistema constrói uma estimativa do estado do mundo x que minimiza a tensão estrutural entre a observação e as expectativas derivadas do modelo interno.

Isso é matematicamente equivalente à inferência variacional em modelos generativos: minimizar uma divergência entre uma distribuição posterior $p(x|m)$ e uma

aproximação variacional $q(x)$.

Memória como Reconstrução

Na perspectiva de MEM | 8, memória não é recuperação de um estado armazenado. É reconstrução de um estado a partir de uma sequência de eventos.

Isso tem uma consequência importante: memórias são necessariamente incompletas e potencialmente inconsistentes. Ao reconstituir um estado passado, o sistema está interpolando sobre regiões do espaço de estados que nunca foram observadas diretamente.

Essa perspectiva é consistente com o que sabemos sobre memória humana: memórias são reconstruções ativas, não recuperações passivas. O sistema nervoso central não armazena experiências como arquivos; reconstrói experiências a partir de fragmentos, contexto e expectativas.

Para sistemas de IA, isso sugere que a questão relevante não é *quantas* memórias um sistema pode armazenar, mas *quão fielmente* pode reconstituir estados passados a partir de representações comprimidas.

Raciocínio como Simulação

Na perspectiva RSVP, raciocínio é a evolução do estado interno de um sistema em direção a configurações que minimizam tensões estruturais. Resolver um problema é encontrar um estado que satisfaz um conjunto de restrições simultaneamente.

Isso conecta raciocínio a:

Satisfação de restrições (CSP): encontrar atribuições de variáveis que satisfazem um conjunto de restrições.

Programação por restrições: um paradigma de programação onde soluções são encontradas através da propagação e satisfação de restrições.

Inferência em modelos probabilísticos: encontrar distribuições que minimizam divergências em relação a observações e priors.

Consciência como Projeção e Correção

A hipótese mais especulativa, mas também a mais interessante, é que consciência — ou algo funcionalmente equivalente a ela em sistemas artificiais — emerge do processo contínuo de projeção e correção: o sistema projeta um modelo do mundo, recebe feedback sobre as diferenças entre o modelo e o mundo, e corrige o modelo iterativamente.

Esse processo não é apenas computação sobre dados. É computação sobre a própria representação do sistema de si mesmo e do mundo que habita — uma forma de auto-referência que pode ser a base de comportamentos que parecem, de fora, dotados de intencionalidade genuína.

Sistemas que atualizam continuamente seus modelos internos com base em discrepâncias entre previsões e observações — que têm, em algum sentido, um “loop de atenção” sobre sua própria representação do mundo — exibem uma forma de aparente intencionalidade emergente que não requer consciência no sentido filosófico forte, mas que é funcionalmente indistinguível dela para observadores externos.

O Núcleo Unificado: Admissibilidade

O Conceito Central

Ao longo dos frameworks apresentados, um conceito aparece repetidamente sob diferentes nomes: o espaço de estados admissíveis, o conjunto de acessibilidade, as restrições que organizam o sistema, o subconjunto de projeções consistentes.

Esse conceito pode ser unificado sob a noção de *admissibilidade*: a estrutura que define o que é possível dentro de um sistema.

Definição 6 (Espaço de Admissibilidade). *Para um sistema com espaço de estados S , o espaço de admissibilidade $\mathcal{A} \subseteq S$ é o subconjunto de estados que satisfazem todas as restrições ativas do sistema. Uma transição $s \rightarrow s'$ é admissível se $s' \in \mathcal{A}$ e se existe um caminho admissível de s a s' .*

Admissibilidade como Geometria

A perspectiva mais rica é tratar admissibilidade geometricamente: o espaço de admissibilidade $\mathcal{A}$ é uma variedade (ou sub-variedade) no espaço de estados S , com sua própria geometria induzida pelas restrições.

Operações sobre o sistema — transições de estado, aprendizado, inferência, raciocínio — são movimentos sobre essa variedade. A curvatura da variedade codifica as interações entre restrições. Regiões de alta curvatura correspondem a restrições que interagem fortemente; regiões de baixa curvatura correspondem a restrições independentes.

Essa perspectiva conecta os frameworks apresentados a:

Geometria da informação: que estuda a curvatura do espaço de distribuições de probabilidade.

Geometria riemanniana em aprendizado de máquina: que usa métricas induzidas pela estrutura dos dados para guiar otimização.

Variedades de dados: a hipótese de que dados de alta dimensão vivem em variedades de baixa dimensão incorporadas no espaço ambiente.

A Pergunta Unificada

Todos os frameworks apresentados podem ser vistos como instâncias de uma única pergunta:

A Pergunta Fundamental. Dado um sistema em um estado atual x , com um espaço de admissibilidade $\mathcal{A}$, qual é o conjunto $R(x) \cap \mathcal{A}$ de futuros acessíveis e admissíveis? E como esse conjunto muda quando as restrições do sistema mudam?

CLIO responde essa pergunta no nível da inferência: o conjunto de estados do mundo consistentes com uma observação.

RSVP responde no nível da dinâmica: os atratores para os quais o sistema converge naturalmente.

MEM|8 responde no nível da memória: os estados que podem ser reconstituídos a partir de uma sequência de eventos.

HYDRA responde no nível da arquitetura: os comportamentos que emergem da coordenação modular.

TARTAN responde no nível da transformação: as propriedades que são preservadas através de decomposições.

Implicações para a Prática

Design de Sistemas

A perspectiva restrição-primeiro tem implicações concretas para o design de sistemas de software:

Especificação antes de implementação: definir o espaço de admissibilidade antes de implementar o sistema. Isso não é simplesmente escrever testes; é especificar formalmente as propriedades invariantes que o sistema deve preservar.

Restrições como documentação: as restrições de um sistema são mais informativas que sua implementação. Uma restrição diz o que o sistema *deve* fazer; uma implementação diz o que ele *faz* em um conjunto específico de condições.

Monitoramento de admissibilidade: sistemas robustos monitoram continuamente se estão operando dentro de seu espaço de admissibilidade e tomam ação corretiva quando detectam violações.

Avaliação de Sistemas de IA

A pergunta mais importante sobre um sistema de IA não é sua acurácia em um benchmark específico. É a forma e o tamanho de seu espaço de admissibilidade: quais

inputs o sistema processa bem, quais ele processa mal, e quais são as fronteiras entre esses regimes.

Um sistema com alta acurácia em inputs in-distribution mas que degrada catastróficamente em inputs out-of-distribution tem um espaço de admissibilidade pequeno em relação ao espaço de inputs possíveis. Esse é o problema da generalização, reformulado geometricamente.

Engenharia de Memória

O framework MEM | 8 sugere uma abordagem à engenharia de memória que vai além do design de índices e estruturas de dados:

Memória como log de eventos: projetar sistemas onde o estado é derivado de um log imutável de eventos, não armazenado diretamente.

Consistência como invariante: tratar a consistência entre réplicas como um invariante que o sistema deve preservar, não como um objetivo a ser alcançado por protocolos de sincronização.

Identidade como trajetória: projetar identidades de entidades em sistemas distribuídos em torno de suas trajetórias, não de seus estados atuais.

Conclusão

Os frameworks apresentados neste ensaio — CLIO, RSVP, MEM | 8, HYDRA e TARTAN — parecem, à primeira vista, díspares. Um trata de inferência, outro de dinâmica, outro de memória, outro de arquitetura, outro de decomposição. Seus domínios de aplicação vão de cosmologia a bancos de dados.

Mas compartilham um núcleo conceitual que os unifica:

A realidade, a cognição e a computação podem ser entendidas como processos de navegação em espaços de possibilidades organizados por estruturas de admissibilidade. O que existe em qualquer instante é apenas a realização atual de um conjunto de possibilidades muito maior. O que importa, para entender o sistema, não é apenas o estado atual mas a estrutura de restrições que define quais estados são possíveis e quais transições são admissíveis.

Essa mudança de perspectiva — de conteúdo para restrição, de estado para trajetória, de distância para acessibilidade — não invalida os modelos e técnicas existentes. Ela os reencadra de uma maneira que torna certas questões mais fáceis de formular e certos problemas mais fáceis de atacar.

Em vez de perguntar apenas *o que existe?*, essa perspectiva nos ensina a perguntar:

O que ainda é possível?

Essa é a pergunta que une física, inteligência artificial, sistemas distribuídos, teoria da informação e cognição sob uma linguagem comum baseada em restrições, aces-

sibilidade, memória e transformação estrutural.

References

- [1] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3–4), 379–423, 623–656.
- [2] Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory*, 2nd ed. Wiley.
- [3] Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos*. Westview Press.
- [4] Holland, J. H. (1995). *Hidden Order: How Adaptation Builds Complexity*. Addison-Wesley.
- [5] Meadows, D. H. (2008). *Thinking in Systems: A Primer*. Chelsea Green.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [7] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [8] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.
- [9] Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. *arXiv preprint arXiv:1312.6114*.
- [10] Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational Inference: A Review for Statisticians. *Journal of the American Statistical Association*, 112(518), 859–877.
- [11] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565.
- [12] Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. *Symposium on Self-Stabilizing Systems*, 386–400.
- [13] Fowler, M. (2005). Event Sourcing. martinfowler.com/eaDev/EventSourcing.html.
- [14] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- [15] Sipser, M. (2012). *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning.
- [16] Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.

- [17] Amari, S., & Nagaoka, H. (2000). *Methods of Information Geometry*. AMS & Oxford University Press.
- [18] Varela, F. J., Thompson, E., & Rosch, E. (1991). *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press.
- [19] Clark, A. (2015). *Surfing Uncertainty: Prediction, Action, and the Embodied Mind*. Oxford University Press.
- [20] Rossi, F., van Beek, P., & Walsh, T. (Eds.). (2006). *Handbook of Constraint Programming*. Elsevier.
- [21] Awodey, S. (2010). *Category Theory*, 2nd ed. Oxford University Press.
- [22] Schacter, D. L. (1996). *Searching for Memory: The Brain, the Mind, and the Past*. Basic Books.
- [23] Clarke, E. M., Grumberg, O., & Peled, D. A. (1999). *Model Checking*. MIT Press.